

TimeGen AI

Automatic College Timetable Generator

Detailed Technical Architecture, File Reference, and Operations Guide

Application type Full-stack web application	Primary stack React + TypeScript + Express + MongoDB
AI capability Google Gemini integration	Prepared for Local development and MongoDB Atlas deployment

Generated on 2026-08-12 from the current TimeGen AI codebase.

1. Executive Summary

TimeGen AI is a full-stack college timetable generator. Administrators create the master data for teachers, classrooms, laboratories, student classes, subjects, assignments, operating hours, breaks, and resource availability. The application then generates a timetable while respecting resource, teacher, class, batch, availability, and capacity constraints.

The system is designed to run locally during development and to persist data either in MongoDB Atlas/local MongoDB or, if MongoDB is unavailable, a local JSON document store. The UI is a React single-page application served through Vite middleware by an Express server.

The project also includes an AI assistant. It sends a selected, current summary of timetable data to Google Gemini and uses the response for natural-language assistance and conflict explanation.

2. High-Level Architecture

Layer	Technology	Responsibility
Browser UI	React 19, TypeScript, Tailwind CSS, Motion, Lucide, Recharts	Renders pages, forms, tables, timetable views, charts, dialogs, and AI chat.
API server	Node.js, Express, TypeScript	Exposes REST API endpoints, validates/normalizes requests, orchestrates scheduling and storage.
Scheduling engine	Custom TypeScript solver and validator	Creates time slots, builds theory/practical activities, allocates rooms/labs, and reports conflicts.
Persistence	MongoDB driver; JSON fallback	Uses MongoDB when MONGODB_URI connects; otherwise uses timegen_local_db.json in the project root.
AI layer	@google/genai	Calls Gemini for assistant replies and conflict explanations when GEMINI_API_KEY is present.

Request flow: React page -> src/services/api.ts -> Express route in server.ts -> database/scheduler/AI module -> JSON response -> React state/UI.

3. Frontend

The frontend is a React single-page application. It is compiled by Vite and uses TypeScript for safer data handling. The browser calls same-origin endpoints under /api, so no separate frontend server address is needed in local development.

Technology	How it is used
React 19 and react-dom	Component and page rendering.
TypeScript	Shared interfaces describe API payloads, entities, schedules, and validation results.
Vite + @vitejs/plugin-react	Development transformation and production frontend build.
Tailwind CSS 4 + @tailwindcss/vite	Styling and responsive layouts.
lucide-react	Navigation, action, status, and domain icons.
motion	User-interface animations.
recharts	Dashboard and analytics visualizations.

Key frontend behaviour

The Timings page lets the administrator choose working days, opening hours, period duration, and named break start/end times. The page presents a single daily preview even though the scheduling API returns slots for every working day.

The Availability page presents a unique row for each period. It avoids duplicate display rows by taking one representative slot per period index. It supports teachers, classrooms, labs, and student classes.

Practical hours are constrained in the assignment forms to 0 or multiples of 2. Each 2-hour practical is treated as one continuous lab session.

4. Backend and Scheduling

The Express server is the application entry point. It loads .env through dotenv/config, creates the API routes, initializes storage, seeds data if needed, and serves Vite in development or static assets in production.

Component	Responsibility
server.ts	Express app, REST routes, assignment normalization, startup, API aggregation, Vite/static serving.
scheduler/solver.ts	Creates time slots, batches, activities, and timetable entries. It prefers compact schedules while checking conflicts.
scheduler/validator.ts	Detects resource, teacher, batch, class, capacity, availability, and workload issues. Calculates quality score.
database/store.ts	Storage abstraction. Reads/writes MongoDB collections or local JSON fallback using the same async API.
seed/demoData.ts	Builds initial classrooms, labs, classes, teachers, subjects, assignments, timings, and generated timetable.
ai/assistant.ts	Gathers context and uses Gemini for assistant chat and conflict explanations.

Scheduling rules implemented

Rule	Implementation
Theory allocation	One period in a classroom with enough capacity.
Laboratory allocation	A practical requires a lab. Each practical block occupies two consecutive non-break periods.
Practical hour rule	Valid practical hours are 2, 4, 6, etc. 2 hours equals one lab session on one day.
Batch handling	A class can be split according to lab bench capacity. Batch practicals may use different labs in parallel.
Resource conflicts	The same teacher, room, lab, class, or batch cannot occupy an overlapping period.
Availability	Blocked teacher/classroom/lab periods are skipped during allocation.
Breaks	Slots that overlap a configured break are treated as non-teaching slots.

5. Database and Data Model

MongoDB is the preferred production storage. The database name defaults to `timegen_ai`, and is explicitly configurable through `DATABASE_NAME`. MongoDB Atlas is supported through a `mongodb+srv` connection string in `.env`. If the connection fails or `MONGODB_URI` is absent, the app continues with `timegen_local_db.json` in the project root.

Never commit `.env` files. They contain the MongoDB URI and Gemini API key. The `.gitignore` excludes `.env` and `.env.*` while retaining `.env.example` as a template.

MongoDB collection / local JSON key	Stores
classrooms / classrooms	Room name, room number, capacity, building, floor, availability metadata.
labs / labs	Lab code, type, bench capacity, location, staff metadata.
student_classes / classes	Class details, student count, and generated batches.
teachers / teachers	Faculty identity, department, contact details, daily workload limit.
subjects / subjects	Subject master data, code, type, description.
class_subjects / classSubjects	Class-subject-teacher assignment, theory/practical weekly hours, lab and batch requirement.
time_slots / timingConfig	One timing configuration document with working days, period length, and breaks.
resource_availability / availabilityRules	Unavailable teacher, room, lab, or class periods.
timetable_entries / timetableEntries	Generated or manually edited timetable sessions, resource, lock status, and duration.
generation_runs / generationRuns	Generation quality, warnings, conflicts, and timing information.

Environment configuration

`MONGODB_URI` points to MongoDB Atlas or local MongoDB. `DATABASE_NAME` selects the database (recommended: `timegen_ai`). `GEMINI_API_KEY` enables AI functions. `APP_URL` is optional and supports hosted/callback scenarios.

6. AI / LLM Integration

The project uses the Google Gen AI JavaScript SDK package `@google/genai`. The configured model name in `backend/ai/assistant.ts` is **gemini-3.6-flash**. This is the LLM used by the application; no OpenAI model is used by this codebase.

Function	How it works
<code>processAIChat()</code>	Reads current resources and selected timetable entries from storage, builds a concise context prompt, appends conversation history, and calls Gemini.
<code>explainConflictAI()</code>	Sends a conflict description to Gemini, requests JSON, and returns an explanation plus suggested resolutions.
Fallback behaviour	If the API key, network, or model call fails, backend returns a local fallback answer/suggestions so the UI remains usable.
Data shared with LLM	Selected counts, resources, classes, teachers, subjects, and a small timetable sample. It does not send the full database by default.

Security note

Treat Gemini API keys and MongoDB URIs as secrets. Keep them only in `.env` or deployment secrets, rotate a password that has been exposed, and restrict Atlas network access to trusted IP addresses.

7. Local Development and Production

Prerequisite: Node.js 18+ with npm. The application runs one Express process on port 3000; that process hosts both the API and the development frontend.

PowerShell commands:

```
Set-Location E:\timegen-ai
npm install
npm run dev
```

Open `http://localhost:3000`. For a production build: `npm run build`, followed by `npm run start`.

8. Complete Source File Reference

This section maps every source/configuration file in the current project tree to its role. Generated folders such as node_modules and dist are intentionally excluded.

Path	Purpose
server.ts	Main Express server, API endpoints, database initialization, generation API, and development/production serving.
package.json	Project metadata, dependencies, and npm scripts: dev, build, start, clean, lint.
package-lock.json / bun.lock	Exact dependency lock files for npm and Bun respectively.
.env.example	Safe environment-variable template. Copy to .env and add real secrets.
.gitignore	Prevents secrets, node_modules, build outputs, and logs being committed.
index.html	HTML entry document used by Vite.
vite.config.ts	Vite plugins, root alias, and development HMR/watch configuration.
tsconfig.json	TypeScript compiler configuration.
metadata.json	Project metadata used by the original app environment.
README.md	Human-readable project description and run guidance.
src/main.tsx	Browser entry point that mounts the React application.
src/App.tsx	Top-level application shell and page routing/state selection.
src/index.css	Global CSS and Tailwind styling declarations.
src/types.ts	Shared domain types: entities, timing, availability, timetable, reports, and AI messages.
src/services/api.ts	Typed browser API client for all /api REST endpoints.
src/components/Navigation.tsx	Application navigation/sidebar.
src/components/AIChatDrawer.tsx	AI assistant chat interface.
src/components/ConfirmModal.tsx	Reusable confirmation dialog for destructive actions.

Path	Purpose
src/pages/DashboardPage.tsx	Overview dashboard: key counts, high-level timetable status, and quick actions.
src/pages/TeachersPage.tsx	Teacher create/read/update/delete management.
src/pages/SubjectsPage.tsx	Subject master management.
src/pages/ClassroomsPage.tsx	Classroom create/read/update/delete management.
src/pages/LabsPage.tsx	Laboratory create/read/update/delete management, including bench capacity.
src/pages/ClassesPage.tsx	Student class management, batch calculation, and class-specific subject assignments.
src/pages/AssignmentsPage.tsx	Global class-subject-teacher assignments; practical hours use two-hour increments.
src/pages/TimingsPage.tsx	Working days, college hours, period duration, user-defined breaks, and daily preview.
src/pages/AvailabilityPage.tsx	Teacher/classroom/lab/class availability matrix with one row per period.
src/pages/GeneratePage.tsx	Starts timetable generation and presents run status.
src/pages/TimetablePage.tsx	Timetable visualization, filtering, entry management, and locking.
src/pages/ConflictsPage.tsx	Displays validation conflicts and AI-assisted explanations.
src/pages/AnalyticsPage.tsx	Timetable metrics and charts, including theory/practical views.
src/pages/ExportPage.tsx	Exports timetable information (for example CSV).
src/pages/SettingsPage.tsx	Administrative data actions such as demo seeding and clearing.
backend/database/store.ts	MongoDB/local JSON data-access layer and collection mapping.
backend/scheduler/solver.ts	Time-slot creation, batching, allocation, practical-block scheduling, and run summary.
backend/scheduler/validator.ts	Conflict and warning validation plus quality-score calculation.
backend/seed/demoData.ts	Sample college dataset and initial generated schedule.
backend/ai/assistant.ts	Gemini client, AI chat, context construction, and conflict explanation.
timegen_local_db.json	Local fallback database created/updated when MongoDB is not used.

9. Operational Checklist

Task	Check
Before first run	Install Node.js, run npm install, create .env, choose Atlas or local JSON storage.
For MongoDB Atlas	Create database user, allow current IP in Network Access, set MONGODB_URI, then restart npm run dev.
Before generation	Enter rooms/labs, teachers, classes, subjects, assignments, timings, breaks, and availability.
For laboratory assignments	Set practical hours to 2/4/6 and choose a lab. Confirm bench capacity supports the generated batch size.
After changes	Regenerate and review conflicts, warnings, and the timetable. Lock entries that must not move.

End of guide